Failure handling (three errors, three reactions) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Failure handling` section — the same file also holds the Basic exercise, under a `# Basic` section. Run it with `cd practice && python custom_errors_practice.py`.

Builds on: the `# Basic` section of that same file — same habit, three error classes instead of one. Nothing to copy or import: scroll up in the file you already have.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

You need 3 different custom error classes, one function that can raise any of the 3 depending on its input, and a caller with 3 *separate* `except` blocks — not one shared one. The whole point is that each error type gets its own, different reaction, not just its own printed message.

Think of a real example: a function processing an order might fail because the item is out of stock, the payment was declined, or the address is invalid — three genuinely different problems that a caller should handle differently.

- Define 3 classes, each inheriting from `Exception`: e.g. `OutOfStockError`, `PaymentDeclinedError`, `InvalidAddressError`.
- One function takes an input (like a status code, or a string describing what went wrong) and raises the matching error.
- A `try/except` block can have multiple `except` clauses, checked in order: `except OutOfStockError: ... except PaymentDeclinedError: ... except InvalidAddressError: ...`
- Each `except` block should do something *different* — not just print a different string, but actually react differently (like retrying, versus asking the user to fix something, versus just logging and stopping).

Hint 2 — The plan, and almost the whole thing

```
define OutOfStockError, PaymentDeclinedError, InvalidAddressError,
    each as a kind of Exception

function process_order(problem_type):
    if problem_type is "stock":
        raise OutOfStockError
    if problem_type is "payment":
        raise PaymentDeclinedError
    if problem_type is "address":
        raise InvalidAddressError

for each of the 3 problem types:
    try to process_order with that type
    except OutOfStockError: print "try again later, notify the warehouse"
    except PaymentDeclinedError:
        print "ask the customer for a different payment method"
    except InvalidAddressError: print "ask the customer to fix their address"
```

```
# practice/custom_errors_practice.py — Failure handling section
class OutOfStockError(Exception):
    pass

class PaymentDeclinedError(Exception):
    pass

class InvalidAddressError(Exception):
    pass

def process_order(problem_type: str) -> None:
    if problem_type == "stock":
        raise OutOfStockError("item is out of stock")
    # add the other two raises yourself
```

Try finishing the function and the 3-branch `except` block yourself before checking the Solution.

Full solution: Show me the solution